

Relatório Técnico e Documentação da POC: Notificações via Slack API e Java

Este documento registra formalmente a Prova de Conceito (POC) realizada para validar o módulo de disparo de notificações corporativas integrando a API do Slack a uma aplicação desenvolvida em Java. O projeto contempla desde o provisionamento de credenciais na plataforma de desenvolvedores do Slack até a identificação e correção de restrições de arquitetura e segurança (CORS e organização de classes em Java).

1. Objetivo da Prova de Conceito

Validar a viabilidade técnica de envio programático de mensagens para um canal específico no Slack utilizando requisições HTTP nativas em Java, servindo de base arquitetural para a integração acadêmica de notificações (Slack e JavaMail).

2. Dados e Configurações de Ambiente

Parâmetro	Valor / Descrição	Finalidade
Workspace	growthmap-grupo07.slack.com	Ambiente de testes do projeto
Channel ID	C0BV8BMQ8RG	Identificador único do canal de destino
OAuth Scope	chat:write	Permissão para o bot postar mensagens em canais
Endpoint API	https://slack.com/api/chat.postMessage	Endpoint REST responsável pelo envio da notificação
Método HTTP	POST	Envio de payload JSON com cabeçalho de autorização Bearer

3. Passo a Passo da Configuração na Plataforma Slack

1. **Criação do App:** Criação de um aplicativo do tipo "From scratch" associado ao workspace de desenvolvimento.
2. **Definição de Escopos (Scopes):** No menu *OAuth & Permissions*, adição do escopo `chat:write` na seção *Bot Token Scopes*.
3. **Instalação no Workspace:** Instalação autorizada para gerar o **Bot User OAuth Token** (prefixo `xoxb-`).
4. **Entrada no Canal:** Convite manual do Bot para o canal de testes digitando `/invite @NomeDoBot` dentro do canal `#testes`, evitando o erro de permissão `not_in_channel`.

4. Desenvolvimento e Problemas Identificados

Tentativa com Front-end (HTML / JavaScript) e Bloqueio de CORS

Inicialmente, foi criado um arquivo HTML com interface simples para disparar a notificação via `fetch()` no navegador. No entanto, o console retornou os seguintes erros:

```
Access to fetch at 'https://slack.com/api/chat.postMessage' from origin 'null' has been blocked by CORS policy: Request header field authorization is not allowed by Access-Control-Allow-Headers in preflight response.
```

Causa Técnica: A API do Slack adota medidas rígidas de segurança contra vulnerabilidades do tipo Cross-Origin Resource Sharing (CORS). A plataforma não permite que chamadas contendo credenciais sensíveis (cabeçalhos `Authorization: Bearer xoxb-...`) sejam realizadas a partir do navegador ou de arquivos locais (`file:///`). Essa regra visa evitar o vazamento inadvertido de tokens em scripts de front-end.

5. Implementação Final Homologada em Java

A solução adotou o cliente nativo `java.net.http.HttpClient`, disponível a partir do Java 11, eliminando a dependência de bibliotecas de terceiros e operando no servidor (livre de bloqueios de CORS).

```
package backEnd;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class NotificacaoJava {
    public static void main(String[] args) throws Exception {
        // Substituir pelo token ativo do bot (xoxb-)
        String token = "xoxb-SEU-TOKEN-AQUI";
        String channel = "C0BV8BMQ8RG";
        String texto = "Mensagem de teste da POC - Notificações Slack!";
```

```

String payload = "{\"channel\":\"" + channel + "\",\"text\":\"" + texto + "\"}";

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://slack.com/api/chat.postMessage"))
    .header("Content-Type", "application/json; charset=utf-8")
    .header("Authorization", "Bearer " + token)
    .POST(HttpRequest.BodyPublishers.ofString(payload))
    .build();

HttpClient client = HttpClient.newHttpClient();
HttpResponse<String> resposta = client.send(request, HttpResponse.BodyHandlers.ofString());

System.out.println("Código HTTP: " + resposta.statusCode());
System.out.println("Retorno Slack: " + resposta.body());
}
}

```

6. Recomendação Crítica de Segurança

Atenção: Tokens com prefixo xoxb concedem privilégios diretos de escrita e leitura no workspace. Como uma das chaves transitou em código de teste durante as etapas anteriores, recomenda-se acessar api.slack.com/apps > OAuth & Permissions e efetuar a rotação (Revoke / Reinstall) da credencial.

7. Conclusão e Próximas Etapas

- **Validação da POC:** A comunicação direta entre a aplicação Java e a API do Slack foi validada com sucesso, apresentando código HTTP 200 e payload {"ok":true}.
- **Integração com JavaMail:** Com o canal do Slack homologado, a próxima fase consiste em desacoplar a camada de serviço para permitir o envio híbrido ou alternativo via e-mail corporativo utilizando o protocolo SMTP.